solderbench: An open-source synthetic benchmark for PCB solder-joint defect inspection algorithm development

Qinmin Wang[a,1], Zixin Wang[b,2], Hongyuan Wang[a], Zongzhong Wang[c,*]

[a] *School of Computer Science and Artificial Intelligence, Guangdong University of Science and Technology, Dongguan, China*
[b] *School of Electronic and Communication Engineering, Tianjin Normal University, Tianjin, China*
[c] *School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, China*

Keywords: solder joint inspection; automated optical inspection; 3D point cloud; synthetic benchmark; defect classification; reproducible evaluation; PCB; synthetic dataset

1. Required metadata

Table 1: Code metadata (mandatory).

Nr.	Code metadata description	Value
C1	Current code version	v0.1.0
C2	Permanent link to code/repository used for this code version	[TO BE FILLED: permanent GitHub rep
C3	Permanent link to reproducible capsule	[TO BE FILLED: Zenodo archival DOI, e
C4	Legal code license	MIT Licer
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python (>=
C7	Compilation requirements, operating environments and dependencies	numpy, scipy, pandas, scikit-learn, matplotlib; OS-ind
C8	Link to developer documentation/manual	[TO BE FILLED: developer documentatio
C9	Support email for questions	wangzz@gdut

2. Motivation and significance

Surface-mount technology (SMT) dominates modern printed circuit board (PCB) assembly, and the quality of solder joints largely determines the reliability of the final product. Automated optical inspection (AOI) is the standard quality-control workhorse on SMT lines [1, 2]. Classical AOI operates on 2D images and has been studied for three decades [3], and deep-learning-based 2D detectors now achieve high accuracy on public image datasets such as DeepPCB [4, 5]. Increasingly, however, solder-joint inspection is moving to 3D: whether a joint contains too little solder, too much solder, or has collapsed around an internal void is fundamentally a volumetric question, and 3D AOI cameras that recover joint geometry as point clouds are now common on production lines [6, 7]. This shift creates a methodological problem. While general-purpose 3D deep learning has advanced rapidly — with point-cloud architectures such as PointNet and PointNet++ [8, 9] and mature tooling such as Open3D [10] — research on 3D solder-joint inspection is severely hampered by the absence of public data. Existing public PCB inspection datasets are almost entirely 2D image collections (e.g., DeepPCB [5]); the few published 3D solder-joint studies all rely on

*Corresponding author at School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, China. E-mail: wangzz@gdut.edu.cn
[1] E-mail: wangqinmin@gdust.edu.cn
[2] E-mail: 396514839@qq.com

private, non-downloadable in-house point clouds, so there is no standardized, reproducible 3D solder-joint benchmark. Real 3D scans come from production lines and are commercially sensitive; manufacturers rarely release them, and the few published 3D point-cloud studies rely on private in-house data [6, 7]. As a consequence, (i) new algorithms cannot be compared against each other on a common footing, (ii) reported numbers are difficult to reproduce, and (iii) researchers without access to an SMT line — including most academic groups and students — are effectively excluded from the problem. The 2D side of the field solved an analogous problem with public benchmarks [5]; no equivalent exists for 3D solder joints. solderbench addresses this gap with a synthetic-data route. Instead of relying on hard-to-release real-world production scans — industrial PCB scans are difficult to anonymize meaningfully, since board layouts and components themselves are proprietary — it simulates solder-joint point clouds from a parametric geometric model whose parameters (nominal joint size, noise level, defect strength, bridging distance, collapse depth, and so on) are physically interpretable and user-controllable. Synthetic generation has three practical advantages over the real-data route. First, cost: a labeled dataset of arbitrary size can be generated in seconds on a laptop, with perfect labels by construction and no confidentiality constraints. Second, control: class distributions, difficulty, and sensor noise can be dialed deliberately, which enables controlled stress tests that real data cannot provide. Third, reproducibility: every cloud is a deterministic function of a random seed, so two researchers running the same seed obtain point-wise identical data. Concretely, solderbench supports three scientific questions: (i) which geometric descriptors separate the defect classes and how much each contributes; (ii) how classical machine-learning pipelines compare with the rule-based methods used in industry (e.g., S3DA); and (iii) how algorithm rankings shift as data difficulty varies. By fixing generation, features, baselines, and protocol in one dependency-light package, it turns anecdotal comparisons into a reproducible benchmark. We stress that solderbench is intended for fair algorithm comparison and descriptor-level analysis — not to certify industrial deployment accuracy (see Section 6, Limitations).

3. Software description

3.1. Software architecture

solderbench is a pure-Python package (about 1,000 lines of non-comment source code, excluding blank lines and docstrings) organized as a linear pipeline of five modules with a single public API surface exported from solderbench/__init__.py: generator.py — SolderJointGenerator: the parametric point-cloud simulator (data layer); features.py — extract_features: the seven-dimensional geometric descriptor extractor (representation layer); baselines.py — get_baselines: the six baseline classifiers (model layer); evaluate.py — run_comparison / run_ablation: experiment runners with automatic CSV, booktabs LaTeX, and figure export (evaluation layer); data.py — load_dataset(): a unified data entry point and the designated integration point for real 3D-camera point clouds. Note that real-data integration requires the user to supply already-segmented single-joint point clouds: solderbench does not provide board-level instance segmentation, so a separate upstream step (e.g., connected-components or a learned detector on the raw PCB scan) must run before load_dataset(). The data flow is strictly feed-forward: generator -> feature matrix -> classifiers -> evaluation tables/figures. Every stage consumes and returns plain NumPy arrays or pandas DataFrames, so any stage can be replaced independently — most importantly, synthetic clouds from the generator can be swapped for segmented real scans at the load_dataset() boundary without touching downstream code. The package ships with a unittest suite (15 tests covering dataset shape and class balance, exact same-seed reproducibility, feature validity, a two-sample Welch's t-test (evaluated at the significance threshold set inside the unit-test suite) confirming that insufficient-solder joints have a significantly smaller mean volume than good joints, and fit/predict smoke tests for all six baselines), a pyproject.toml for pip install -e ., and a codemeta.json metadata file. The only runtime dependencies are numpy, scipy, pandas, scikit-learn, and matplotlib.

3.2. Software functionalities

Parametric point-cloud generation. SolderJointGenerator simulates surface-mount solder joints as 3D point clouds in five classes that mirror the dominant defect taxonomy of SMT production and the defect geometry reported for industrial 3D-AOI systems [6, 7]: good — a nominal joint modeled as a half-ellipsoid with natural size variation; insufficient — too little solder, modeled as a uniformly shrunk ellipsoid (default shrink range 0.55-0.85 of nominal scale); excessive — too much solder with an irregular profile, modeled as an inflated ellipsoid (default mean inflation

1.3) plus a random bump of extra points; bridging — a solder bridge to the neighbouring joint, modeled by an abnormally small centre-to-centre distance plus a low strip of bridge points; collapse — a collapsed/void joint, modeled by a Gaussian dent at a random offset of the joint top. More than fifteen parameters are exposed, including the number of points per cloud (n_points), sensor noise amplitude (noise), the insufficient-solder shrink range, the excessive-solder inflation mean/standard deviation and bump size range, the bridging distance mean/standard deviation and bridge point-count range, and the collapse depth range. Class distributions deliberately overlap — for example, the upper tail of insufficient overlaps the lower tail of good — so that no classifier can reach a perfect score; this is a deliberate methodological property that keeps the benchmark discriminative. Overlap is realized by overlapping the per-class parameter ranges (e.g., the insufficient-solder shrink range 0.55–0.85 overlaps the lower tail of the nominal good-joint scale), so users can tune difficulty simply by widening or narrowing these ranges in the generator configuration. **Geometric feature extraction.** extract_features computes seven physically interpretable descriptors per joint: vol (volume by voxel counting, voxel size 0.01), max_h (maximum height), h_var (variance of point heights), area (convex-hull surface area, approximating the joint surface), vol_area_ratio (vol / area), convex_vol_ratio (vol / ConvexHull.volume, a shape-irregularity measure computed with scipy.spatial.ConvexHull), and min_dist (centre-to-centre distance to the nearest neighbouring joint). **Baseline classifiers.** get_baselines returns six baselines spanning

Table 2: The seven geometric descriptors returned by `extract_features` and their physical meaning.

Feature	**Definition**	**Physical meaning**
`vol`	voxel count $\times$ voxel size^3	total solder volume
`max_h`	max z over points	joint height
`h_var`	variance of point heights	height-distribution spread (dent / bump)
`area`	convex-hull surface area	joint surface extent
`vol_area_ratio`	vol / area	compactness
`convex_vol_ratio`	vol / ConvexHull.volume	shape irregularity (voids / dents)
`min_dist`	min distance to neighbouring joint	bridging exposure

the methods a practitioner would try first: S3DA, a re-implementation of the industrial volume-threshold rule [6] in which a joint is classified by thresholding its estimated solder volume (our implementation follows the published thresholding logic without vendor-specific calibration; see baselines.py for the exact threshold; the threshold is fixed in code and users may tune it for custom datasets); logistic regression; a support vector machine with an RBF kernel [12]; k-nearest neighbours; gradient boosting; and a random forest [11]. All trainable baselines are scikit-learn estimators [13] with fixed random states. **Experiment runners.** run_comparison(X, y) trains and evaluates all six baselines under a fixed protocol (stratified 7:3 train/test split plus 5-fold cross-validation on the training set) and returns a ranked DataFrame of accuracy, macro precision, macro recall, macro-F1, and cross-validated accuracy; run_ablation(X, y) performs leave-one-feature-out and single-feature analysis for the random forest. Ablation is anchored on the random forest because it is the strongest and most widely used tabular baseline (Table 3, B6); fixing the model isolates each feature's contribution from model-choice effects, so the deltas reflect descriptor value rather than classifier idiosyncrasy. Both runners export their tables as CSV and booktabs LaTeX, and run_comparison additionally renders the confusion matrix of the best model as a PNG figure. A single command, python examples/run_benchmark.py, reproduces the full benchmark reported in Section 3.

3.3. Implementation details

Two design decisions deserve emphasis because they protect the benchmark's validity. *Prevention of sampling-density leakage.* Voxel-counting volume estimates are biased by point density: a denser cloud fills more voxels and looks 'bigger'. If defect classes were generated with different point counts, a classifier could cheat by learning the sampling density instead of the geometry. solderbench therefore resamples every cloud to exactly n_points points by uniform random subsampling (when the native count exceeds n_points; sparser clouds are handled by the same routine — see generator source), so volumetric features reflect shape, not density. *Seed-level reproducibility.* All randomness flows from a single independent `numpy.random.RandomState` instance created at construction (not the global NumPy seed), so external code that sets NumPy's global seed cannot break reproducibility. The same seed produces

point-wise identical clouds, identical feature matrices, and identical experiment tables; this property is asserted by the test suite. Combined with the fixed evaluation protocol and the pinned random_state of every baseline, the complete benchmark is bit-for-bit reproducible, and the tables in Section 3 can be regenerated with one command. The feature extractor uses scipy.spatial.ConvexHull [15] for surface-area and convex-volume terms; array plumbing is NumPy [14]; tables are pandas DataFrames [16]; figures are rendered with matplotlib [17]. There is no compiled code, no GPU requirement, and no network access at runtime.

4. Illustrative examples

4.1. Quickstart

Install the package and its dependencies first with `pip install -e .` (requires Python $\geq$ 3.8); the API below corresponds to version v0.1.0 and may change in later releases. The following end-to-end example generates a dataset, extracts features, and runs both experiments in eight lines:

```
from solderbench import (SolderJointGenerator, extract_features,
get_baselines, run_comparison, run_ablation)
# generate 1250 joints (5 classes x 250) at seed 42
gen = SolderJointGenerator(n_points=1500, noise=0.02, seed=42)
clouds, labels = gen.generate(n_per_class=250)   # 5 classes x 250 joints
X, y = gen.to_features(clouds, labels)           # 7-feature matrix + labels
results = run_comparison(X, y)   # 6 methods x Acc/P/R/F1 + 5-fold CV
print(results[["no", "name", "Accuracy", "F1"]])
abl = run_ablation(X, y)         # leave-one-out + single-feature dF1
print(abl.head(8))
```

4.2. Benchmark comparison

examples/run_benchmark.py produces the benchmark snapshot reported in Table 3: 1250 samples (5 classes x 250 joints, 1500 points per cloud, seed 42, stratified 7:3 split; 5-fold cross-validation on the training set). Three

Table 3: Baseline comparison on the solderbench benchmark (seed = 42, 1250 samples). Best results in bold.

No.	Method	Accuracy	Precision	Recall	F1	CV Acc. (5-fold)
B1	S3DA (volume threshold)	0.8960	0.8990	0.8960	0.8964	0.9234 +/- 0.0138
B2	Logistic Regression	0.9387	0.9416	0.9387	0.9386	0.9657 +/- 0.0198
B3	SVM (RBF)	0.9387	0.9390	0.9387	0.9385	0.9771 +/- 0.0177
B4	KNN	0.8933	0.8944	0.8933	0.8921	0.9234 +/- 0.0168
B5	GradientBoosting	0.9413	0.9417	0.9413	0.9412	0.9577 +/- 0.0138
B6	**RandomForest baseline**	**0.9440**	**0.9452**	**0.9440**	**0.9439**	**0.9680 +/- 0.0160**

observations follow. First, the industrial rule S3DA (B1) is a strong baseline at 0.8960 accuracy, confirming that solder volume alone carries most of the class signal — exactly the physics the generator encodes. Second, all learning-based methods improve on the rule, with the random forest best on the held-out test set (accuracy 0.9440, macro-F1 0.9439). Third, test-set and cross-validation rankings do not fully coincide: the RBF SVM has the highest CV accuracy (0.9771) yet only 0.9387 on the held-out test set, a textbook signature of mild overfitting to the training folds. This is exactly why solderbench reports both protocols and why users must trust the independent test set rather than cross-validation alone. Figure 1 shows the confusion matrix of the random forest on the held-out test set. Bridging is perfectly separated (75/75), because the min_dist feature exposes the abnormally small inter-joint distance by construction. The dominant residual confusion is between collapse and good/insufficient (4 and 6 samples, respectively): a shallow dent alters the height distribution only subtly, which is precisely why `h_var` — the most informative descriptor in the ablation (Table 4) — is needed to separate it; this mirrors the hard case reported for real 3D inspection systems [6, 7].

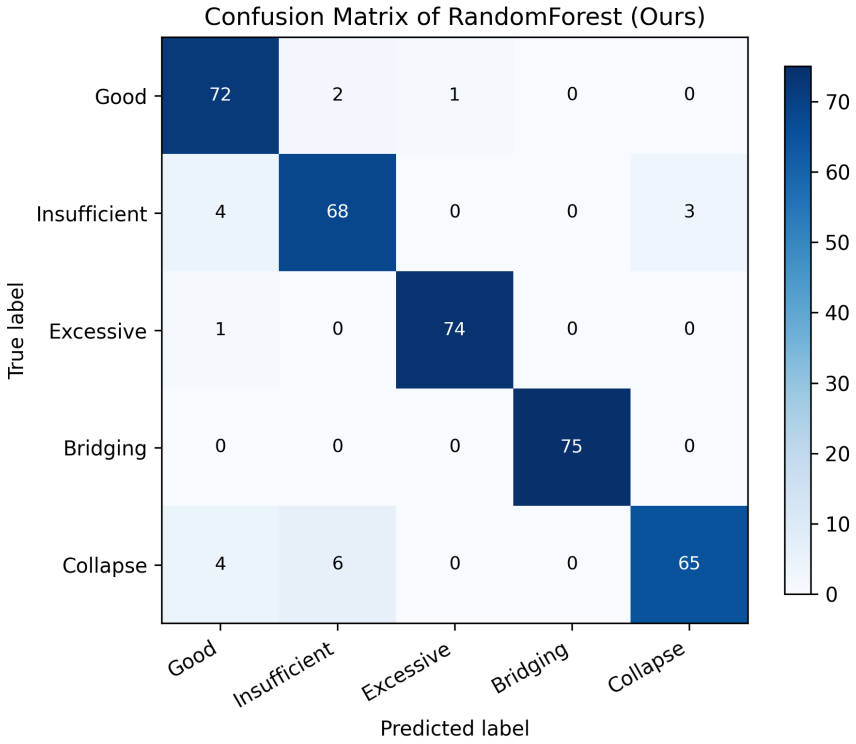


Figure 1: Confusion matrix for the RandomForest baseline (B6) on the held-out test set (375 samples, seed = 42). Bridging is perfectly classified via the min_dist feature; the main confusion occurs between collapse and good/insufficient classes.

4.3. Feature ablation

Table 4 reports the leave-one-feature-out (LOO) ablation for the random forest (full-model macro-F1 = 0.9439). The ablation has a clean physical reading. Height variance (h_var) is the single most valuable descriptor because

Table 4: Leave-one-feature-out ablation of the RandomForest model (macro-F1; full model = 0.9439).

Removed feature	dF1 vs. full model	Interpretation
h_var	-0.0459	Most informative: collapse dents and excessive bumps reshape the height distribution
vol_area_ratio	-0.0105	Compactness cue separating volumetric defects from nominal joints
min_dist	-0.0054	Small overall cost because bridging is nearly separable already
area	-0.0052	Correlated with vol; adds modest independent signal
convex_vol_ratio	-0.0025	Irregularity cue; partially redundant with h_var
vol	+0.0026	Slightly redundant once ratios are present
max_h	0.0000	Fully redundant given h_var and vol

the two hardest defects — collapse (a dent) and excessive solder (a bump) — are primarily height-distribution phenomena, whereas volume-only cues saturate. Single-feature runs corroborate this picture: the strongest single feature is convex_vol_ratio (macro-F1 = 0.6412), which captures shape irregularity common to several defect types, while the weakest is min_dist (macro-F1 = 0.3616), which is highly informative for bridging but carries no signal for the other four classes. The near-zero or positive deltas for vol and max_h indicate information redundancy rather than irrelevance: both are strongly correlated with the ratio features. Volume alone is in fact a strong single cue — it underpins the industrial S3DA rule (B1), which reaches 0.8960 accuracy using volume only — and its near-zero delta here reflects redundancy with the ratio features, not uselessness; removing vol slightly increases macro-F1 within small numerical fluctuations, which further illustrates that volume information is already encoded within ratio-based descriptors. These results demonstrate the kind of descriptor-level insight that a controlled synthetic benchmark makes cheap to obtain and exact to reproduce.

Table 5: Representative single-feature macro-F1 (RandomForest, seed 42). The complete single-feature set is exported by run_ablation.

Single feature	macro-F1
`convex_vol_ratio`	0.6412
`min_dist`	0.3616

5. Impact

solderbench contributes to three communities. For inspection-algorithm research, it provides the first public, reproducible 3D benchmark for solder-joint defect classification, replacing per-paper private datasets with a common task, a common protocol, and versioned reference numbers; new methods can be ranked against six baselines with a single command. For teaching, the package is a self-contained case study in geometric machine learning: the complete pipeline from point-cloud simulation to feature engineering to model comparison fits in a short lab session, runs on any laptop, and every design decision (fixed point count, overlapping class tails, dual evaluation protocols) doubles as a lesson in benchmark methodology. For reproducibility practice, the combination of seed-level determinism, a one-command benchmark script, machine-readable exports (CSV/LaTeX/PNG), a codemeta.json descriptor, and an MIT license makes the software trivially citable, archivable, and reusable. The path from synthetic benchmark to production data is deliberately short. A user with access to a real 3D AOI camera replaces the body of solderbench.data.load_dataset — or simply passes their own X, y — with code that reads the camera scans, segments individual joints, and calls solderbench.extract_features; all downstream experiments run unchanged. The synthetic benchmark can then serve as a pre-training or sanity-check stage, and the feature ablation indicates which descriptors deserve attention when real data are scarce. Beyond the built-in features-plus-classical-ML workflow, the generator's raw point clouds can also be passed directly to external deep point-cloud models (e.g., PointNet [8, 9]), which

learn their own representations and bypass the hand-crafted feature module. solderbench is designed to support both scenarios.

6. Limitations

We state the limitations of solderbench honestly; they define the scope in which its results should be interpreted. The generator is a geometric model, not a physics simulator: it does not model wetting dynamics, surface tension, oxidation, or realistic sensor artifacts such as occlusion, multi-path reflections, or structured-light failure modes, and its noise model is a simple isotropic perturbation. Class-conditional distributions were chosen to be plausible and overlapping, not fitted to measured production data. Three further scope limits should be noted. First, the five defect classes are idealized single-fault geometries; the generator does not currently support compound defects (e.g., a joint that is simultaneously insufficient and collapsed), although such compound faults occur on real lines. Second, the point clouds model only the individual solder joint and omit the surrounding PCB substrate, copper pads, and component pins, so board-level context is out of scope. Third, resampling every cloud to a fixed n_points simplifies real sensors, which return a variable point count that scales with joint geometry. This fixed-count resampling may change the point-density distribution compared with real sensor outputs. **Consequently, solderbench is suitable for algorithm prototyping, fair cross-method comparison, and descriptor-level analysis, but its metric values must not be read as predictions of real industrial AOI performance.** Synthetic validation cannot substitute for validation on real scans; the benchmark guarantees only that comparisons made on it are fair, controlled, and exactly reproducible — the property the field currently lacks.

7. Conclusions

We presented solderbench, an open-source, pure-Python synthetic benchmark for 3D PCB solder-joint defect inspection. It combines a parametric five-class point-cloud generator with seed-level reproducibility, a seven-dimensional geometric feature extractor, six baselines including the industrial S3DA volume-threshold rule, and automated comparison/ablation runners with CSV, LaTeX, and figure export. On the reference benchmark (1250 samples, seed 42), a random forest achieves 0.9440 accuracy and 0.9439 macro-F1, and ablation identifies height variance as the dominant descriptor. By removing the data-access barrier that currently fragments 3D solder-joint inspection research, solderbench makes algorithm development, comparison, and teaching in this domain open and reproducible. Future work includes (i) fitting generator parameters to measured scans as they become available, (ii) adding sensor-artifact models, (iii) extending the class taxonomy to compound defects, (iv) providing user scripts that convert real segmented joint point clouds into the tool's input format, (v) developing input-output interfaces for deep point-cloud models (e.g. PointNet, PointNet++) so raw point clouds generated by solderbench can be directly fed to deep networks without hand-crafted feature extraction, and (vi) shipping preset difficulty templates (high-noise, enhanced class-overlap) for algorithm stress testing.

8. Acknowledgements

[Acknowledgements placeholder — funding, colleagues, and infrastructure to be acknowledged here.]

References

[1] Q. Ling, N.A.M. Isa, Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey, IEEE Access 11 (2023) 15921-15944. https://doi.org/10.1109/ACCESS.2023.3245093

[2] Y. Zhou, M. Yuan, J. Zhang, G. Ding, S. Qin, Review of vision-based defect detection research and its perspectives for printed circuit board, Journal of Manufacturing Systems 70 (2023) 557-578. https://doi.org/10.1016/j.jmsy.2023.08.019

[3] M. Moganti, F. Ercal, Automatic PCB inspection systems, IEEE Potentials 14(3) (1995) 6-10. https://doi.org/10.1109/45.464686

[4] W. Dai, A. Mujeeb, M. Erdt, A. Sourin, Soldering defect detection in automatic optical inspection, Advanced Engineering Informatics 43 (2020) 101004. https://doi.org/10.1016/j.aei.2019.101004

[5] S. Tang, F. He, X. Huang, J. Yang, Online PCB defect detector on a new PCB defect dataset, arXiv preprint arXiv:1902.06197 (2019).

[6] Y. Xu, Z. Wang, J. Liang, Y. Zhang, J. Feng, W. Miao, J. Wang, G. Pu, S3DA: A 3D point cloud based PCB solder defect detection algorithm, in: S. Liu (Ed.), Software Fault Prevention, Verification, and Validation (SFPVV 2024), Lecture Notes in Computer Science, vol. 15393, Springer, 2025, pp. 115-130. https://doi.org/10.1007/978-981-96-1621-3_8

[7] Q. Hu, K. Hao, B. Wei, H. Li, An efficient solder joint defects method for 3D point clouds with double-flow region attention network, Advanced Engineering Informatics 52 (2022) 101608. https://doi.org/10.1016/j.aei.2022.101608

[8] C.R. Qi, H. Su, K. Mo, L.J. Guibas, PointNet: Deep learning on point sets for 3D classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 652-660. https://doi.org/10.1109/CVPR.2017.16

[9] C.R. Qi, L. Yi, H. Su, L.J. Guibas, PointNet++: Deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5099-5108.

[10] Q.-Y. Zhou, J. Park, V. Koltun, Open3D: A modern library for 3D data processing, arXiv preprint arXiv:1801.09847 (2018).

[11] L. Breiman, Random forests, Machine Learning 45(1) (2001) 5-32. https://doi.org/10.1023/A:1010933404324

[12] C. Cortes, V. Vapnik, Support-vector networks, Machine Learning 20(3) (1995) 273-297. https://doi.org/10.1007/BF00994018

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825-2830.

[14] C.R. Harris, K.J. Millman, S.J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N.J. Smith, et al., Array programming with NumPy, Nature 585 (2020) 357-362. https://doi.org/10.1038/s41586-020-2649-2

[15] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261-272. https://doi.org/10.1038/s41592-019-0686-2

[16] W. McKinney, Data structures for statistical computing in Python, in: Proceedings of the 9th Python in Science Conference, 2010, pp. 51-56. https://doi.org/10.25080/Majora-92bf1922-00a

[17] J.D. Hunter, Matplotlib: A 2D graphics environment, Computing in Science & Engineering 9(3) (2007) 90-95. https://doi.org/10.1109/MCSE.2007.55